E4 234 Advanced Power System Analysis — Programming Assignment

Promts and exchanges()

User:

i want to develop a newton raphson load flow solver in python for my project. I already have the network data and ybus part, now i want to develop the actual solver.

I will give the logic to be followed and you generate the codes accordingly. dont make the entire solver in one file, i want it as seperate modules so that i can understand each part and also commit them individually in github.

First identify slack, pv and pq buses from the network data. For slack bus voltage magnitude and angle are fixed, for pv bus voltage magnitude and active power are known and for pq bus both P and Q are specified.

Use NR method in polar form. First calculate P and Q at each bus from Ybus and present voltage values. Then find the mismatch between specified and calculated power. For slack bus no mismatch is required and for PV bus only P mismatch is considered.

After that form the jacobian matrix with H N M L submatrices. Use the actual partial derivative equations, not numerical approximation. The matrix size should change depending on the number of PV and PQ buses.

Then solve for the corrections in voltage angle and magnitude using J delta x = mismatch. use scipy sparse LU for this, dont take inverse of jacobian. Update the voltage and angles after each iteration and repeat untill mismatch is below 10^-8 pu.

Also give provision for changing tolerance and maximum iterations. Initially use flat start, but later i should be able to give initial voltage values also.

Another thing is generator Q limits. If the reactive power required by a PV generator goes above or below its limits, change that bus to PQ and fix Q at the limit. Also check whether it can be changed back to PV when voltage condition is satisfied. This switching should be shown in the iteration log.

The code should work for n bus system, not just for a particular IEEE case. Also consider if multiple generators are connected to the same bus or if any generator or line is out of service.

I need separate files for calculating power, mismatch, jacobian, solving the linear equations, voltage updating and the main NR iteration. You can create additional files if required. Make sure that the output of one function is correctly passed to the next one and the units are consistent.

Finally return the bus voltage magnitudes and angles, P and Q injections, number of iterations, maximum mismatch and convergence status.

Give the codes file by file and explain how they are connected. Also provide test cases for checking the equations and convergence.

start with the solver structure and then proceed module by module.

Chatgpt GO:

I have created the Newton Raphson load flow solver based on the logic and equations you provided. The implementation uses the polar form NR method, with separate modules for each calculation.

The solver is designed to work with your existing network data and Ybus modules. It supports Slack, PV and PQ buses, analytical Jacobian calculation, sparse LU factorization, voltage updates and convergence checking.

The complete solver files are included in the attached ZIP folder:

NR_LoadFlow_Solver.zip *(example attachment name)*

1. Files included

The solver is divided into the following Python files:

File	Purpose
solver.py	Main NR iteration loop and convergence checking
power.py	Calculates active and reactive power injections using Ybus
specifications.py	Forms the specified P and Q injections
mismatch.py	Calculates the power mismatch vector
jacobian.py	Constructs the four Jacobian submatrices H, N, M and L

linear_solver.py	Solves the correction equations using sparse LU
voltage_update.py	Updates voltage magnitudes and phase angles
q_limits.py	Checks generator reactive power limits and performs PV/PQ switching
generator_outputs.py	Calculates the generator active and reactive power outputs

All these modules are kept separately under the src/loadflow/ directory. Additional tests are provided in the tests/ folder.

2. Working of the solver

The program first reads the bus classifications and network data, then obtains the Ybus matrix.

At each iteration, it calculates the active and reactive power injections, forms the mismatch vector and constructs the Jacobian matrix.

The corrections are calculated by solving:

[
J\Delta x=\Delta S
]

using SciPy sparse LU factorization. The voltage magnitudes and angles are then updated.

This process continues until the maximum mismatch is below the specified tolerance or the maximum iteration limit is reached.

The solver returns voltage magnitudes, angles, P and Q injections, iteration count, mismatch history and convergence status.

3. How to add the files to your project

Extract the attached ZIP file and copy the Python modules into your existing project, keeping the folder structure as follows:

E4234_LoadFlow/

│

├── src/

│ └── loadflow/

```
│   ├── models.py
│   ├── ybus.py
│   ├── solver.py
│   ├── power.py
│   ├── specifications.py
│   ├── mismatch.py
│   ├── jacobian.py
│   ├── linear_solver.py
│   ├── voltage_update.py
│   ├── q_limits.py
│   └── generator_outputs.py
│
└── tests/
```

Keep the existing models.py and ybus.py files unless an interface change is required.

4. Running the tests

Open the project folder in VS Code and install the project dependencies:

py -3.14 -m pip install -e .

py -3.14 -m pip install pytest

To run all the automated tests:

py -3.14 -m pytest -q

The tests are intended to check power calculations, mismatch formation, Jacobian construction, voltage updates and convergence behavior.

If any test fails, check the error message and the corresponding module before proceeding.

5. Adding the solver to GitHub

Since you are developing the project module by module, commit each completed module separately.

For example, after adding power.py:

git add src/loadflow/power.py

git commit -m "Implement bus power injection calculation"

Similarly, after adding jacobian.py:

git add src/loadflow/jacobian.py

git commit -m "Implement analytical Newton Raphson Jacobian"

After integrating the main solver:

git add src/loadflow/solver.py

git commit -m "Implement Newton Raphson iteration and convergence"

Use the same procedure for the remaining modules and their associated tests.

You can push the commits to GitHub after completing the modules using:

git push origin master

6. Next step

First run the individual module tests and then test the integrated solver with a small 3-bus system.

After confirming that it converges correctly, we can proceed with IEEE 14, 30, 57 and 118-bus validation using PYPOWER or MATPOWER as the reference.

The reference power flow packages should be used only for validation, not inside the actual NR solver.

User:

i have completed the newton raphson load flow simulator and the basic functions are working. But i am not sure whether the results are completely accurate.

I will upload the zip folder containing the python files. First check the computational modules and then i will provide the entire project including GUI if required.

Go through all the files and check if there are any mistakes in the equations, logic or data being passed between the different modules. Especially check the Ybus formation, power mismatch calculation, jacobian matrix, voltage updates and generator power calculations.

Also check whether there are any unit conversion problems like MW and pu, voltage angles in degrees and radians, transformer tap ratios etc. Make sure that the bus numbering and array indexing are consistent.

Dont just check whether the code runs without errors. I want you to verify whether the results are mathematically correct.

If you find any mistakes, mention which file contains the error, explain why it happens and give the corrected code. Also check whether the problem is in the actual solver or in the data being passed from the GUI.

After correcting the problems, provide the modified files seperately so that i can replace them in my existing github project without changing the other modules.

Also provide some tests to verify the corrections.

Chatgpt Plus:

I examined the computational modules and checked the implementation of the Newton Raphson method.

The main Ybus, power mismatch and analytical Jacobian equations appear to be implemented correctly. However, some issues were identified in network validation, generator voltage regulation, reactive power limit handling and generator output reporting.

The following problems need to be corrected:

1. Generator voltage setpoints are not always consistently applied when the generator setpoint and bus voltage magnitude differ.
2. The network validation does not fully detect disconnected buses or electrical islands.
3. Reactive power limit enforcement can modify the original bus and generator data during PV to PQ switching.
4. Slack bus power may not be included correctly in generator output reporting when an external grid is used.
5. Input values entered through the GUI may cause inconsistencies if physical units and per-unit values are mixed.

The Newton Raphson mathematical equations do not need to be rewritten. The corrections should mainly be made in the data validation, solver initialization, generator handling and related modules.

Corrected files: The updated versions of models.py, solver.py, q_limits.py and generator_outputs.py, together with the necessary tests, should be included in the corrected project package.

After replacing the modified files, run the automated tests and compare the results with reference power flow software.

I examined the computational modules and checked the implementation of the Newton Raphson method.

The main Ybus, power mismatch and analytical Jacobian equations appear to be implemented correctly. However, some issues were identified in network validation, generator voltage regulation, reactive power limit handling and generator output reporting.

The following problems need to be corrected:

1. Generator voltage setpoints are not always consistently applied when the generator setpoint and bus voltage magnitude differ.
2. The network validation does not fully detect disconnected buses or electrical islands.
3. Reactive power limit enforcement can modify the original bus and generator data during PV to PQ switching.
4. Slack bus power may not be included correctly in generator output reporting when an external grid is used.
5. Input values entered through the GUI may cause inconsistencies if physical units and per-unit values are mixed.

The Newton Raphson mathematical equations do not need to be rewritten. The corrections should mainly be made in the data validation, solver initialization, generator handling and related modules.

Corrected files: The updated versions of models.py, solver.py, q_limits.py and generator_outputs.py, together with the necessary tests, should be included in the corrected project package.

After replacing the modified files, run the automated tests and compare the results with reference power flow software.

User:

now i want to verify the accuracy of the corrected solver. Use the IEEE 14, 30, 57 and 118 bus test systems.

Compare the voltage magnitudes, voltage angles, generator P and Q, branch power flows and total system losses with a reference power flow solver like PYPOWER.

Create a validate.py file which automatically runs all four test cases and prints the maximum errors and pass or fail result for each parameter.

Use the same network data and solver settings for both calculations. Also check the number of iterations and maximum mismatch.

Dont use PYPOWER inside my actual solver, it should only be used for reference calculation.

Give the required validation file and explain how to run it from VS Code.

Chatgpt:

The validation script has been prepared to compare the Newton Raphson solver against PYPOWER using the same MATPOWER case data.

Run the following commands from the project directory:

py -3.14 -m pip install PYPOWER

py -3.14 validate.py

The script compares voltage magnitudes, relative voltage angles, generator outputs, branch power flows and total losses.

During validation, an important difference was observed in the IEEE 118 bus system. The reference solver used a slack bus angle of approximately 30 degrees, while the developed solver used 0 degrees.

Since the power flow equations depend on voltage angle differences, the comparison was corrected to use angles relative to the slack bus.

The final validation results were:

Test system	Result
IEEE 14-bus	PASS

IEEE 30-bus PASS

IEEE 57-bus PASS

IEEE 118-bus PASS

All four test systems agreed with PYPOWER within the specified numerical tolerances.

The automated tests should also be run after any future modifications:

py -3.14 -m pytest -q

This confirms the numerical correctness of the tested cases, although GUI input handling and additional operating conditions should still be checked separately.

User:

now the newton raphson solver is working, i want to develop a GUI for this similar to ETAP or PowerFactory.

I want a single line diagram where the user can create a power system network without entering all the data directly in python.

Use PySide6 for developing the GUI. There should be a main canvas where we can place buses, generators, loads, transmission lines, transformers and shunts. Also include an external grid option for slack bus.

The elements should be available in a toolbar or element palette. User should be able to select an element and place it on the canvas. For lines and transformers, connection should be made by selecting two buses.

The buses should be movable and the connected lines should move along with them. Also provide zoom in, zoom out, pan, delete and selection options.

When i select any element its parameters should be shown in a properties panel. For example for a line i should be able to enter R, X, charging B and rating. For transformer include impedance, tap ratio and phase shift. For generators include P, voltage setpoint and reactive power limits.

There should be a Run NR button which takes the network created in GUI and passes it to the existing solver. Dont rewrite the solver for GUI. Use the same computational modules which we already developed.

After running load flow show the voltage magnitude and angle near each bus. Also show branch power flows and loading. I want separate result tables for buses, branches and generators.

The GUI should also show whether the solver converged or not and the number of iterations.

Keep the GUI code separate from the computational modules. You can use QGraphicsView and QGraphicsScene for drawing the single line diagram.

Generate the necessary files like window.py, diagram.py and any other supporting modules. Explain how the GUI data is converted into network objects and passed to the solver.

Give the code module by module so that i can add it to my existing github project.

Chatgpt:

The GUI structure has been developed using PySide6, with the numerical solver kept separate from the interface.

The main application uses QMainWindow, while the single line diagram is created using QGraphicsView and QGraphicsScene.

The GUI source files are included in the project attachment.

The main files are:

File	Purpose
gui/window.py	Main application window, menus, toolbars, properties panel and result tables
gui/diagram.py	Single-line diagram elements, drawing, connections and graphical interactions
gui/__main__.py	Entry point for launching the application
gui/__init__.py	Python package initialization

The interface is connected to the existing network model and Newton Raphson solver.

When the user edits an element, its parameters are updated in the corresponding network object. When Run NR is selected, the GUI passes the network data to the solver and receives the calculated bus voltages, branch flows and generator outputs.

To launch the GUI from the project directory:

py -3.14 -m loadflow.gui

If the required packages are missing, install them using:

py -3.14 -m pip install -e .

After verifying that the GUI launches and the basic components work, add the files to Git:

git add src/loadflow/gui/

git commit -m "Implement PySide6 single line diagram GUI"

The numerical solver files should remain unchanged unless an interface mismatch is identified.

User:

the basic GUI is working now but i want to add more features.

First add an option to save the network as a project file and reopen it later. It should save all the bus, line, transformer, generator and load data along with their positions on the diagram. Use JSON format for this.

Also add undo and redo. If i add or delete an element or change its parameters i should be able to undo the operation. Give shortcuts like ctrl z and ctrl y.

For the properties panel show the units properly. Include the option to enter line impedance in ohms or pu. For transformers add nameplate rated MVA, rated voltage, percentage impedance and X/R ratio and convert them into system base pu.

I also want to select the load model as constant PQ, constant impedance or ZIP from the GUI.

Add study settings like tolerance, maximum iterations, flat start or previous solution, Q limit enforcement and global load scaling.

For results, add a convergence plot and voltage limit checking. Buses having voltage below 0.95 pu or above 1.05 pu should be highlighted. Also highlight branches when loading exceeds 80% and 100%.

Include options to export results as CSV and Excel and generate PDF or HTML reports.

Also provide MATPOWER import and export, IEEE CDF import and transformer tap side settings.

Make sure these new features dont affect the existing numerical calculations. Give only the files which are modified and explain which files i need to replace.

Chatgpt:

The requested GUI features have been added as separate modules while retaining the existing Newton Raphson solver.

The updated files are provided in the corresponding module attachments.

The major changes include:

- JSON project saving and reopening.
- Undo and redo history for network editing.
- Physical-unit and per-unit conversion in the properties panel.
- Constant PQ, Z and ZIP load selection.
- Study settings and convergence plotting.
- Voltage and branch loading violation indicators.
- CSV, Excel, PDF and HTML output.
- MATPOWER and IEEE CDF import/export support.
- Transformer tap-side and tap-range configuration.

The additional supporting files include project_io.py, units.py, reporting.py, matpower_import.py, matpower_export.py and ieee_cdf_import.py.

After copying the modified files, run:

py -3.14 -m pytest -q

py -3.14 validate.py

The IEEE validation should be rerun after integrating the changes to confirm that the numerical results remain consistent.

Launch the GUI and manually verify saving, reopening, element properties, diagram editing and result exports.

Once the functionality is confirmed, commit the changes:

git add .

git commit -m "Add project management and advanced GUI features"

For the final repository, individual module commits should be retained rather than combining all development into a single commit.

User:

I checked the GUI code and identified the problems.

The zoom-in shortcut was using Ctrl++, which may not be recognized consistently on keyboards where the plus symbol requires Shift. An alternative Ctrl+= shortcut was added.

The delete function was also restricted to elements classified as buses. This prevented other graphical elements from being deleted.

The deletion logic has been updated to support buses, generators, loads, shunts, lines and transformers, including multiple selected elements.

Corrected files provided:

- src/loadflow/gui/window.py
- src/loadflow/gui/diagram.py

Replace these two files in the existing project.

Launch the application:

py -3.14 -m loadflow.gui

Test zoom in/out, deletion of individual elements, multiple selection and undo/redo.

After confirming that the corrections work, commit them using:

git add src/loadflow/gui/window.py src/loadflow/gui/diagram.py

git commit -m "Fix zoom shortcuts and element deletion"

The numerical solver is unchanged by these GUI corrections.

Chatgpt:

I checked the GUI code and identified the problems.

The zoom-in shortcut was using Ctrl++, which may not be recognized consistently on keyboards where the plus symbol requires Shift. An alternative Ctrl+= shortcut was added.

The delete function was also restricted to elements classified as buses. This prevented other graphical elements from being deleted.

The deletion logic has been updated to support buses, generators, loads, shunts, lines and transformers, including multiple selected elements.

Corrected files provided:

- src/loadflow/gui/window.py
- src/loadflow/gui/diagram.py

Replace these two files in the existing project.

Launch the application:

py -3.14 -m loadflow.gui

Test zoom in/out, deletion of individual elements, multiple selection and undo/redo.

After confirming that the corrections work, commit them using:

git add src/loadflow/gui/window.py src/loadflow/gui/diagram.py

git commit -m "Fix zoom shortcuts and element deletion"

The numerical solver is unchanged by these GUI corrections.